NP-completeness

Reminder

P: ”easy to solve” – can solve in polynomial time.

NP: ”easy to check yes answers” – certificates with a sound, complete, polynomial-time verifier.

Polynomial-time **reduction**: $L_1 \rightarrow L_2$ – a polynomial-time computable function f such that $l \in L_1$ if and only if $f(l) \in L_2$.

Intuition: an algorithm to decide membership of L_1 given access to a “library” that decides membership of L_2.

Reductions

Suppose we have a reduction $L_1 \rightarrow L_2$.

"Use it for good": If we can solve L_2 we can solve L_1.

"Use it for evil": If L_1 is hard then L_2 is hard.

L is **NP-hard**: for every problem $M \in NP$ there is a polynomial-time reduction $M \rightarrow L$.

If L is NP-hard and also $L \in NP$ then L is **NP-complete**.

What do you think of this problem?

SUBSET-SUM

Input: an integer k, and a multiset of integers $S = \{x_1, x_2, \ldots, x_n\}$.

Question: does there exist a subset $T \subseteq S$ such that $\sum T = k$?

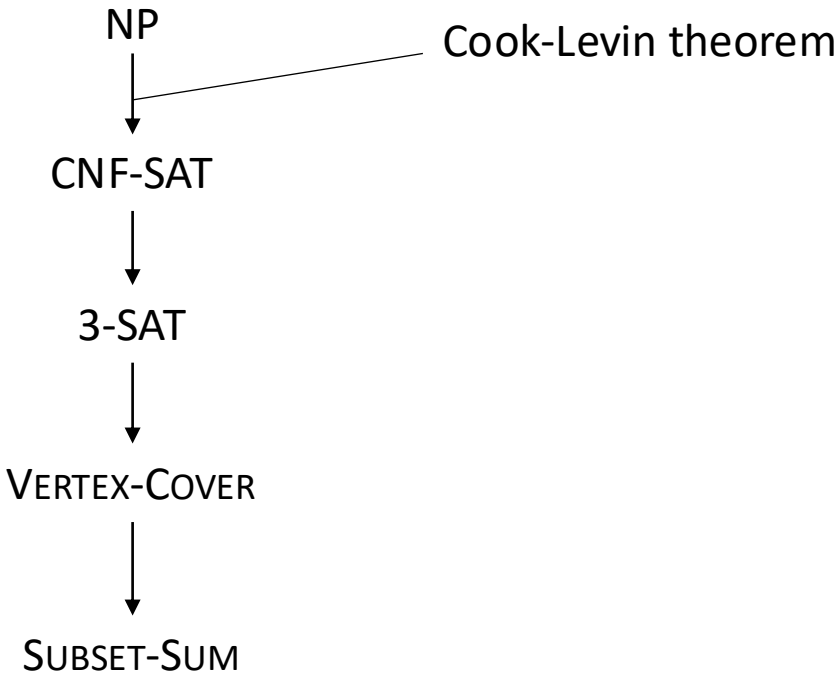
e.g. 5,{1,4,7}: YES, 10,{1,4,7}: NO

Seems mostly harmless.

After thinking for a bit: damn, I can't seem to solve it.

OK we can't solve it, so can we prove it's NP-hard?

Eventually…



Remark: If problem A reduces to problem B and problem B reduces to problem C, then problem A reduces to problem C.
(Exercise: write down a proof of this)

The Cook-Levin theorem

- The hardest bit is showing our first problem is NP-complete – we have to show there's a reduction from *every* problem in NP.
- Once that's done things get better – showing a reduction from *any* known NP-complete problem is enough (why?).
- We'll show a problem called "CNF-SAT" is NP-complete.

Boolean formulae

A Boolean formula is built out of Boolean variables, with negation, addition and multiplication. Addition means disjunction (OR), multiplication means conjunction (AND). A formula is *satisfiable* if there is an assignment of the variables that makes it true.

E.g. $x_1x_2 + \neg x_1\neg x_2$ says the two variables are either both true or both false.

Disjunctive Normal Form (DNF): the formula is the sum of clauses of the form $a_1a_2 \ldots a_k$.

Conjunctive Normal Form (CNF): the formula is the product of clauses of the form $a_1 + \cdots + a_k$, where each a_i is some x_j or $\neg x_j$.

e.g. $(x_1 + \neg x_2)(\neg x_1 + x_2)$

CNF-SAT

CNF-SAT

Input: a Boolean formula ϕ in CNF.

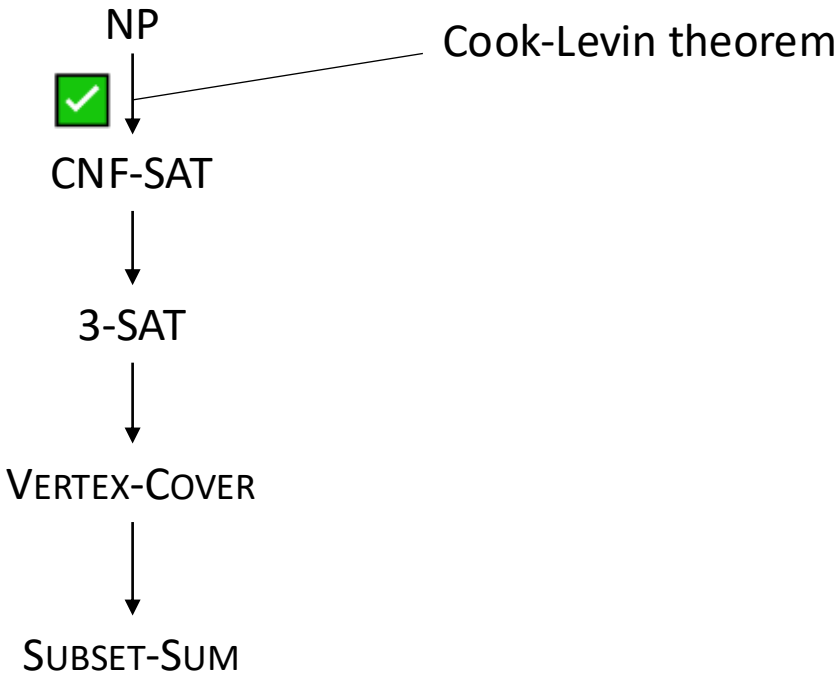
Question: is ϕ satisfiable?

Cook-Levin (sketch)

- Fix an NP language L, so I have a verifier V for checking certificates (formally, V is a "Turing Machine").
- Given an input l, how can I make a formula ϕ which is satisfiable iff $l \in L$?
- To start with I'll have some variables for the bits of the witness w.
- Now we also have to encode the running of the verifier.
- I'll also have a variable $x_{i,t}$ for the contents of memory cell i at time-step t. (And more for the state and position of the read head)
- Fact (painful): we can encode the steps of the machine in a formula.

OK we can’t solve it, so can we prove it’s NP-hard?

Eventually…



3-SAT

3-SAT

Input: a CNF formula ϕ where each clause contains exactly 3 variables.

Question: is ϕ satisfiable?

Example: $(\neg z + x + w)(\neg z + y + w)(z + \neg x + \neg y)$

Non-example: $(x_1 + x_4 + \neg x_9 + x_{11})$

Reducing CNF-SAT to 3-SAT

Our job is to take a formula ϕ and make a formula ψ such that:

- ψ is a 3-SAT formula
- ψ is satisfiable if and only if ϕ is

We will do it clause-by-clause. (We will have to introduce some new variables)

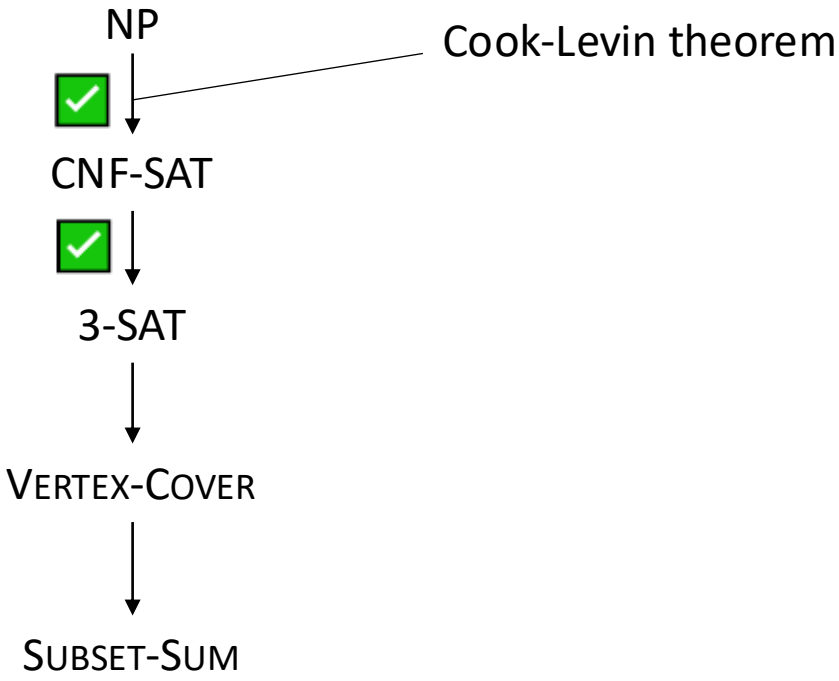
If our clause is:

- $a_1 + a_2 + a_3$: great, include as-is!
- $a_1 + a_2$: replace with $(a_1 + a_2 + b)(a_1 + a_2 + \neg b)$, for new variable b
- a_1: replace it with $(a_1 + b_1 + b_2)(a_1 + b_1 + \neg b_2)(a_1 + \neg b_1 + b_2)(a_1 + \neg b_1 + \neg b_2)$, for new variables b_1, b_2
- $a_1 + a_2 + \cdots + a_k$ $(k > 3)$: replace it with $(a_1 + a_2 + b_1)(\neg b_1 + a_3 + b_2)(\neg b_2 + a_4 + b_3) \ldots (\neg b_{k-3} + a_{k-1} + a_k)$

So this proves that CNF-SAT reduces to 3-SAT.

OK we can’t solve it, so can we prove it’s NP-hard?

Eventually…

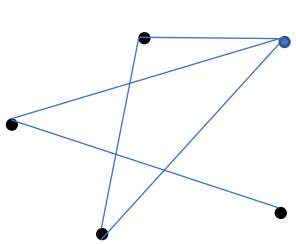


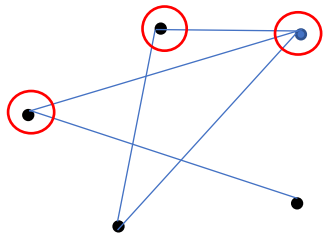
Vertex-Cover

VERTEX-COVER

Input: an integer k and a graph G.

Question: does there exist a set X of k vertices of G such that every edge has at least one of its endpoints in X?





Reducing 3-CNF to Vertex-Cover

Our job is: given a 3-CNF formula ϕ, come up with a graph G and integer k such that G has a k-vertex-cover if and only if ϕ is satisfiable.

We will do this using *gadgets*.

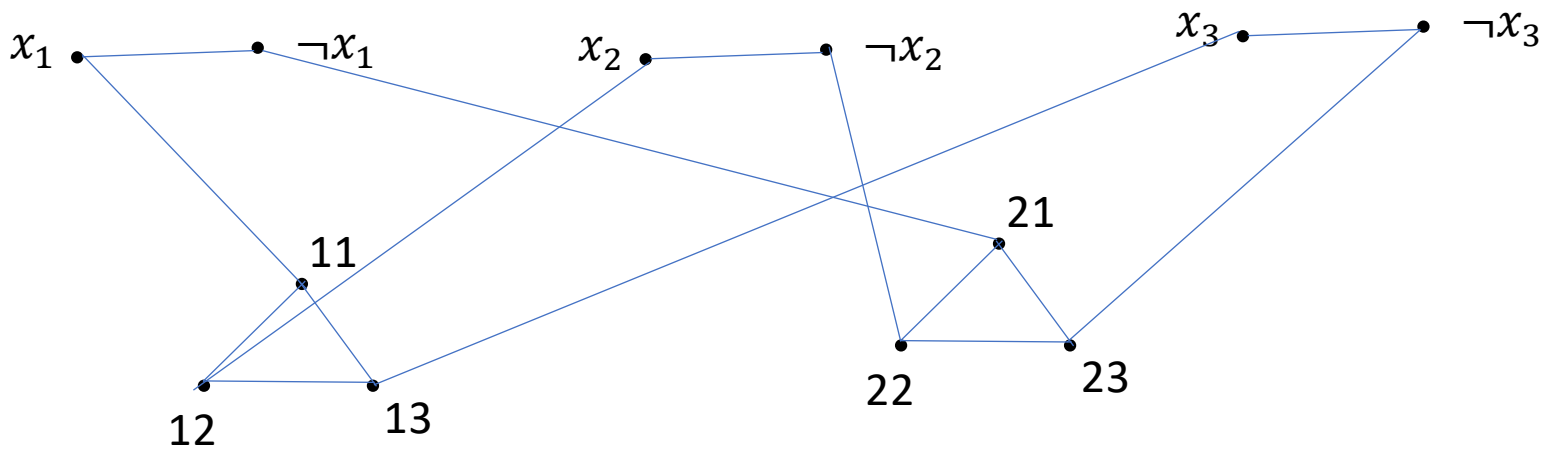
For each variable x_i, add two vertices, labelled x_i and $\neg x_i$ (of course the labels are just in our head), and draw an edge between them.

This means our vertex cover will have to contain vertex x_i or vertex $\neg x_i$.

Reducing 3-CNF to Vertex-Cover

Now for each clause (say the jth clause), add three vertices in a triangle labelled j1, j2 and j3. If our clause is $(a_1 + a_2 + a_3)$, connect j1 to the 'variable-vertex' labelled a_1, j2 to the one labelled a_2 and j3 to the one labelled a_3.

For example, say our formula is $(x_1 + x_2 + x_3)(\neg x_1 + \neg x_2 + \neg x_3)$

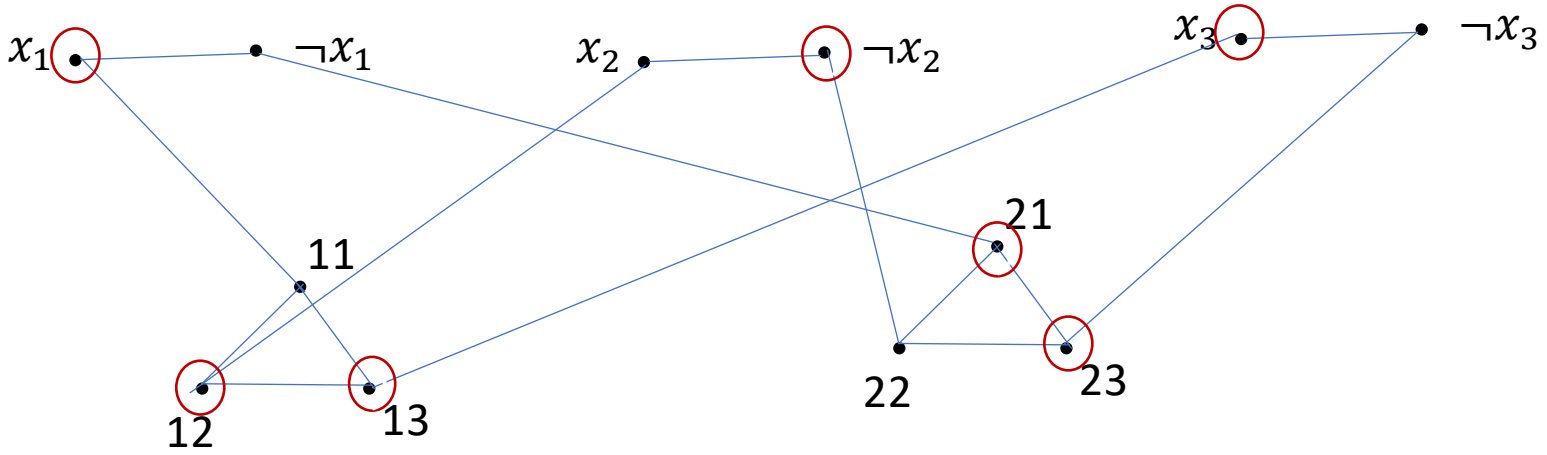


Now set k=m+2n (where our formula has m variables and n clauses)

Reducing 3-CNF to Vertex-Cover

- We need to pick one of each of our m pairs of ‘variable’ vertices
- To cover each of our n triangles, we have to pick two of the three vertices.
- So to have an m+2n vertex cover, the cross-edges have to be covered without taking any more vertices.
- Think of the chosen ‘variable’ vertices as an assignment.
- Then for each clause/triangle, the edge from the not-chosen vertex must be to a chosen variable-vertex.
- Hence that term in the clause is satisfied.
- So every clause in the formula is satisfied, so the formula is true with this assignment.

$(x_1 + x_2 + x_3)(\neg x_1 + \neg x_2 + \neg x_3)$



Reducing 3-CNF to Vertex-Cover

Formally, we have two things to prove:

1. If ϕ is satisfiable then our graph has an m+2n vertex cover
2. If our graph has an m+2n vertex cover then ϕ is satisfiable

1. Take a satisfying assignment, for each $x_i, \neg x_i$ pair pick the one that is true in this assignment. Then for each triangle find a vertex whose cross-edge is to an already picked vertex (exists since the clause is satisfied) and pick the other two vertices.

Reducing 3-CNF to Vertex-Cover

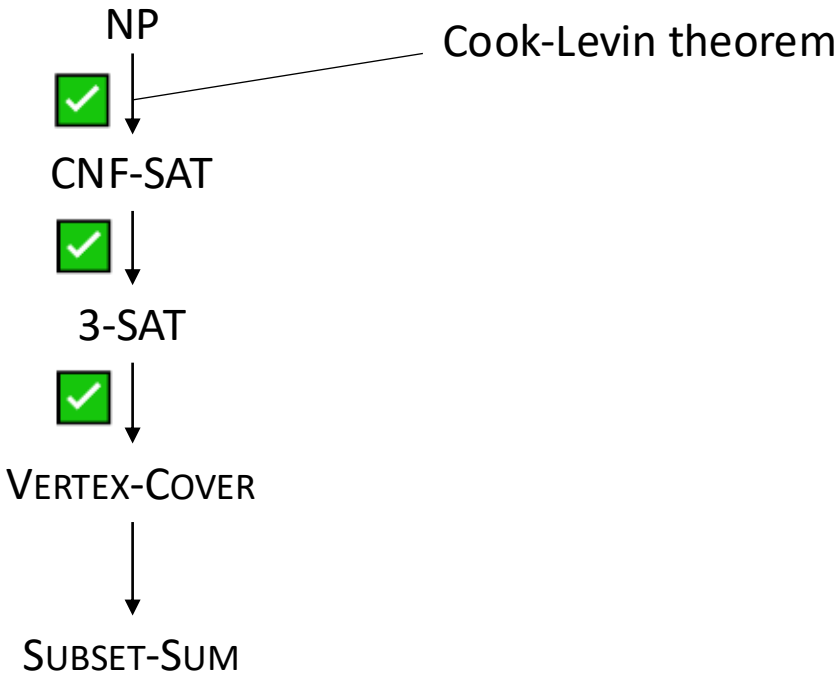
Formally, we have two things to prove:

1. If ϕ is satisfiable then our graph has an m+2n vertex cover
2. If our graph has an m+2n vertex cover then ϕ is satisfiable

2. Take an m+2n vertex cover. This must include at least one from each of the m $x_i, \neg x_i$ pairs, and at least two from each of the n triangles. But that's m+2n vertices, so there must be exactly one of each $x_i, \neg x_i$ and exactly two from each triangle. So take the assignment where x_i is true if vertex x_i is chosen and false if $\neg x_i$ is chosen. Now for each clause, in the corresponding triangle there is an un-chosen vertex. The cross-edge from this vertex must be to a chosen vertex, so the clause is satisfied.

OK we can't solve it, so can we prove it's NP-hard?

Eventually…



Reducing Vertex-Cover to Subset-Sum

SUBSET-SUM

Input: an integer k, and a multiset of integers $S = \{x_1, x_2, \ldots, x_n\}$.

Question: does there exist a subset $T \subseteq S$ such that $\sum T = k$?

Our job is to take a graph G and a number k_1 and make a multiset S and a number k_2 such that S has a subset with sum k_2 if and only if G has a k_1 vertex cover.

Say our graph has n vertices and m edges.

Reducing Vertex-Cover to Subset-Sum

We will have an integer a_i for each vertex v_i. Our first job is to make sure we choose exactly k_1 of them

$a_i = 4^{m+1}$ + [smaller stuff] $\quad\quad$ $k_2 = k_1 4^{m+1}$ + [smaller stuff]

How do we represent the edges?

Write $H(i,j) = \begin{cases} 1 \text{ if vertex i is an endpoint of edge j} \\ 0 \text{ otherwise} \end{cases}$

Reducing Vertex-Cover to Subset-Sum

$$k_2 = k_1 4^{m+1} + \sum_{j=1}^{m} 2 \cdot 4^j$$

$$a_i = 4^{m+1} + \sum_{j=1}^{m} 4^j H(i,j)$$

$$b_j = 4^j, j = 1, \dots, m$$

$$S = \{a_1, \dots, a_n, b_1, \dots, b_m\}$$

Reducing Vertex-Cover to Subset-Sum

- The sum for k_2 has a term for each edge, $2 \cdot 4^j$
- The b_j gives us **one** copy of 4^j, which we can take or not
- So at least one of the chosen a_i must have a 4^j, i.e. have $H(i,j) = 1$
- (and of course at most two can have this since edge j only has two endpoints)
- So indeed S has a k_2-subset if and only if G has a k_1 vertex cover.

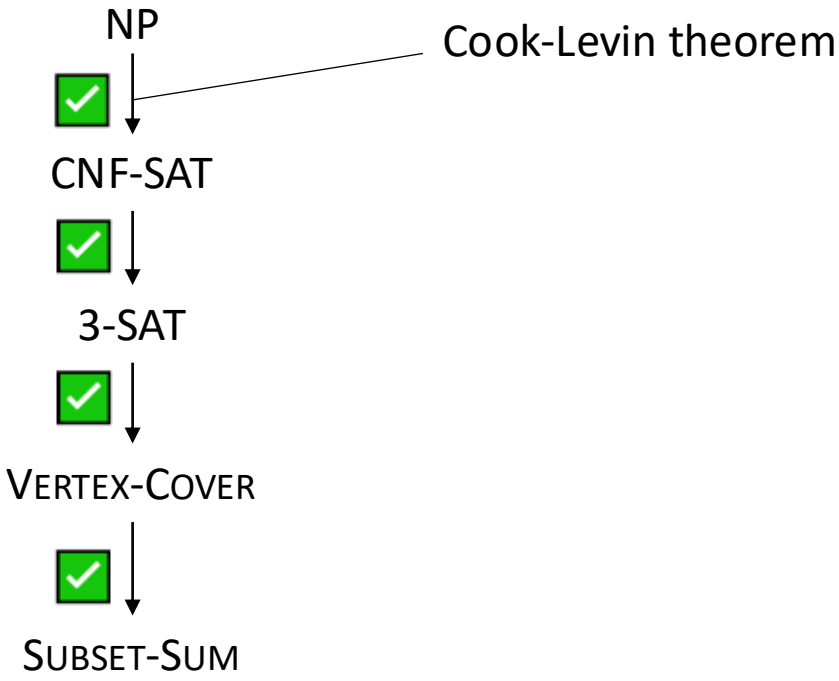
Reducing Vertex-Cover to Subset-Sum

So we have proved:

Theorem: Vertex-Cover is NP-hard (and clearly in NP, so is NP-complete)

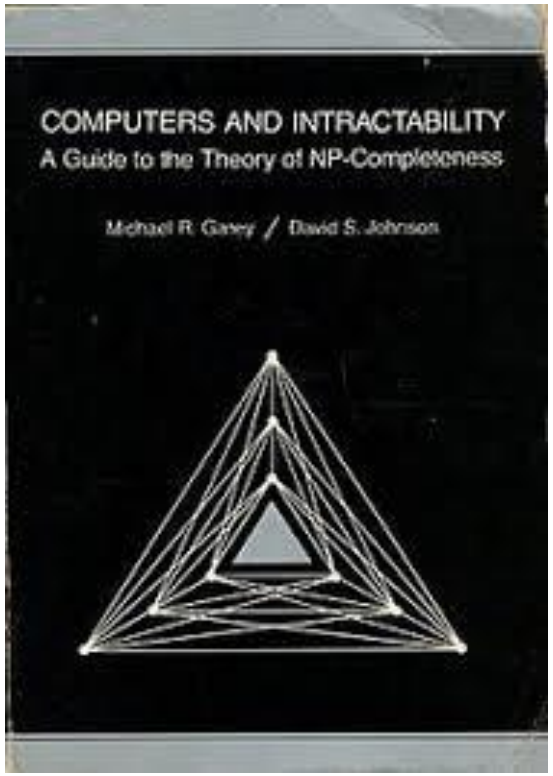
OK we can’t solve it, so can we prove it’s NP-hard?

Eventually…



NP-completeness

In everyday life, we don’t usually have to go through such a long chain of reductions.



We look for a known NP-complete problem that seems similar to the problem we have.

Some examples

HAMILTONIAN-CYCLE

Input: a graph G

Question: does G have a Hamiltonian cycle? (cycle visiting each vertex exactly once)

TRAVELLING SALESMAN PROBLEM (TSP)

Input: a complete weighted graph G and an integer k

Question: does G have a Hamiltonian cycle of weight at most k?

Some examples

CLIQUE

Input: a graph G and an integer k

Question: does G contain a k-clique? (k vertices with every pair joined by an edge)

3-COLOURING

Input: a graph G

Question: can G be 3-coloured? (give each vertex one of 3 colours such that no vertices of the same colour are joined by an edge)

Some examples

INDEPENDENT-SET

Input: a graph G and an integer k

Question: does G contain a k-independent-set? (k vertices with no edges between them)

Let's say we know that is CLIQUE NP-complete. Show that INDEPENDENT-SET is NP-complete.

Some examples

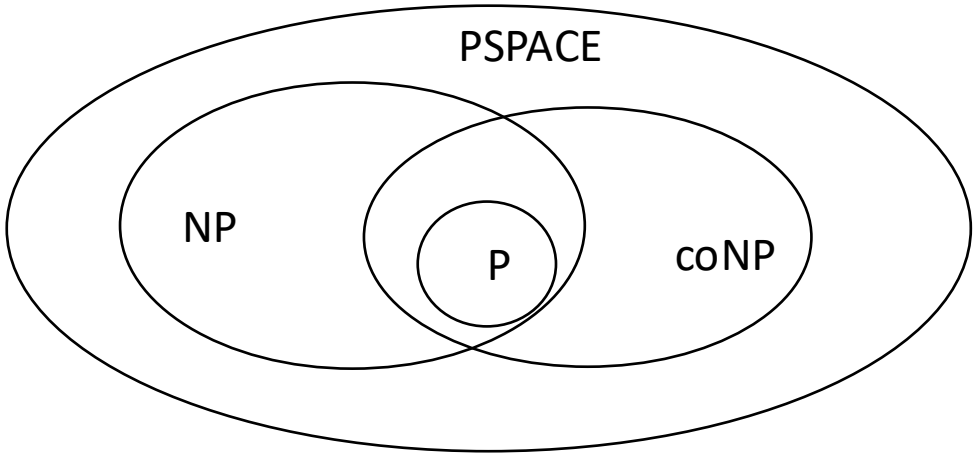
KNAPSACK

Input: a capacity C and a multiset of objects each with a weight w_i and a value v_i

Question: what is the maximum total value of a collection of the objects with total weight at most C?

Other complexity classes

- Remember both our definitions of NP treated ‘yes’ very differently from ‘no’
- NP=“yes answers can be verified”
- We can also think about problems where no answers can be verified
- This is called coNP
- We think the Venn diagram looks like this:



Thoughts on hardness reductions

- This is something people often find challenging.
- Spend some time thinking about what's going on and make sure you understand it conceptually, i.e. why a reduction $L_1 \to L_2$ plus L_1 NP-hard implies L_2 NP-hard.
- **Reducing L_1 to L_2 means specifying a function from L_1 instances to L_2 instances, not just giving an analogy between L_1 and L_2 ("the edges in L_2 represent the roads in L_1…")**
- How to think of the reduction? Unfortunately there's no real recipe
- Practice, practice, practice. (And come to tutorials! We're here to help)

Exercises

R-17.3,17.4,17.5,17.9.

C-17.9,17.11,17.12,17.13,17.17

A-17.2,17.4,17.5,17.6

CLRS 34.5-2,34.5-3,34.5-7,34.5-8, 34-2, 34-3, 34-4.